

COS 487 / 587 Course Project Part 1: IR Rangers

Esther Greene
University of Southern Maine
Portland, Maine, USA
esther.greene@maine.edu

Courtney Jackson
University of Southern Maine
Portland, Maine, USA
courtney.jackson@maine.edu

Nathaniel Serrano
University of Southern Maine
Portland, Maine, USA
nathaniel.serrano@maine.edu

Abstract

...

Keywords

information retrieval, ...

1 Introduction

The test collection consists of 34,308 arXiv papers published in August and September 2024. Each paper is provided both as raw HTML scraped from the arXiv website and as a processed JSON file containing its identifier, title, abstract, and paragraphs. Because the JSON files were produced with rule-based extraction, they contain noise such as leftover $\LaTeX$ markup and merged words, which we discuss in Section 2.1. In total, the collection contains 2,275,574 paragraphs, or about 66 per paper.

We address two tasks over this collection: *paper retrieval*, which returns papers identified by their arXiv ID (in YYYY.NNNNN format), and *paragraph retrieval*, which returns individual paragraphs. For each query, systems return their top 100 results. Three query sets are provided: Train (2,000 queries), Study (200 queries with relevance judgments, which we use for all analysis in this paper), and Test (200 queries whose judgments are withheld). Each set contains 30% easy, 50% medium, and 20% hard queries. Relevance is graded on a three-point scale: a document that fully addresses the query is relevant (2), one that provides useful information without fully addressing it is partially relevant (1), and any document not in the judgments is considered non-relevant.

2 Methodology

2.1 Preprocessing

Documents and queries pass through the same preprocessing pipeline, so that query terms match the terms stored in the index. All settings are read from a YAML configuration file, and a copy of the configuration is saved with the processed collections; queries are tokenized with that saved copy at search time. The final pipeline applies the following steps in order:

- (1) **Text cleaning.** Inline mathematics delimited by dollar signs, remaining $\LaTeX$ commands (e.g., $\mathcal{a}$), URLs, and numeric citation markers such as “[5, 6, 7]” are each replaced with a space.
- (2) **Lowercasing.**
- (3) **Tokenization.** Text is split into runs of letters and digits. Underscores act as separators, so a subscript such as x_i yields x and i .
- (4) **Token filtering.** Tokens shorter than two characters and tokens consisting only of digits are discarded.
- (5) **Stopword removal.** Stopwords are frequent function words, such as “the” and “and”, that say little about a document’s

topic. We use NLTK’s English list [1], extended with terms common in scientific writing: “et”, “al”, “fig”, “figure”, “table”, “eq”, and “section”.

- (6) **Stemming.** Remaining tokens are reduced to stems with the Snowball English stemmer [2], so that forms such as “model” and “models” match.

The cleaning and filtering steps were motivated by corpus statistics. With only lowercasing and stopword removal, the 50 most frequent tokens were dominated by mathematical notation rather than words: digits (“1”, “2”, “0”), single-letter variables (“n”, “x”, “k”), underscores from subscripts, and $\LaTeX$ commands such as “mathcal” and “mathbb”. Adding these to the stopword list would also remove real words such as “text” and “prime”, so we instead remove inline math as a whole. A lone dollar sign, such as in a price, would be paired with a math delimiter and the text between them deleted, so math spans are only removed from paragraphs with an even number of dollar signs. Only 1,458 paragraphs (0.06%) contain an unpaired dollar sign. They keep their math spans, although $\LaTeX$ command removal and the token filters still remove most of the resulting tokens.

Table 1 compares three settings. *Baseline* applies only lowercasing, tokenization on letters, digits, and underscores, and stopword removal (with “not” and “no” retained). *Cleaned* applies steps 1–5 and no longer retains “not” and “no”: bag-of-words models cannot use them to express negation, and “not” alone accounted for over 500,000 tokens. + *Snowball* is the final pipeline. Cleaning reduced the collection from 168.0 million to 114.9 million tokens, a 31.6% decrease, and all of the 50 most frequent tokens became words, such as “model”, “data”, and “using”. It also reduced the number of paragraphs longer than 1,000 tokens from 868 to 117; the longest Baseline paragraphs were collaboration author lists made up largely of $\LaTeX$ markup and ORCID identifiers. Stemming leaves the number of tokens unchanged but shrinks the vocabulary by a further 10.7%, from 659,861 to 589,251 terms. The cost is that some distinct words share a stem: “different” and “difference” both become “differ”, and “generate”, “generation”, and “generative” all become “generat”. Finally, cleaning removes every token from 8,389 paragraphs, which consisted mostly of mathematical notation; we describe how these are handled in Section 2.2.

Table 1: Paragraph collection statistics under three preprocessing settings, defined in Section 2.1.

	Baseline	Cleaned	+ Snowball
Total tokens	167,965,275	114,907,087	114,907,087
Unique terms	781,405	659,861	589,251
Terms appearing once	367,415	297,672	272,983
Median length	58	40	40
90th pct. length	143	104	104
Longest paragraph	32,070	20,818	20,818
Over 1,000 tokens	868	117	117
Empty paragraphs	429	8,389	8,389
Under 10 tokens	82,390	230,782	230,782

2.2 Document Representation

2.3 Search Interface

2.4 BM25

2.5 TF-IDF

2.6 Evaluation Setup

3 Evaluation Results

3.1 Metrics

3.2 Efficiency

4 Results Analysis

5 Conclusion

References

- [1] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python*. O'Reilly Media.
- [2] Martin F. Porter. 2001. Snowball: A Language for Stemming Algorithms. <https://snowballstem.org/texts/introduction.html>.